Nivel de ajuste de un modelo y validación cruzada

M6: Aprendizaje de máquina supervisado

|**AE2**: Aplicar técnicas de validación cruzada para la prevención de sobreajuste en los modelos utilizando el concepto de nivel de ajuste óptimo.

alkemy

Introducción👩‍💻

En el mundo del aprendizaje de máquina, la construcción de un modelo predictivo no termina cuando el algoritmo ha sido entrenado. Una de las etapas más cruciales y, a menudo, subestimadas, es la evaluación del modelo y la comprensión de qué tan bien se ajusta a los datos. Los conceptos de ajuste, validación y evaluación son fundamentales para garantizar que un modelo no solo funcione bien en los datos conocidos, sino también en datos nuevos y no vistos.

El ajuste de un modelo hace referencia al grado en que la estructura y los parámetros del modelo representan la relación real entre las variables. Un ajuste inapropiado puede llevar a dos problemas comunes: el sobreajuste y el subajuste. Ambos fenómenos afectan directamente la capacidad del modelo para generalizar, es decir, para predecir correctamente sobre datos nuevos. Por esta razón, es necesario incorporar técnicas que permitan validar y medir el comportamiento del modelo bajo diferentes escenarios y conjuntos de datos.

En este manual abordaremos los fundamentos teóricos y prácticos sobre el ajuste de modelos y la validación cruzada. Aprenderás los tipos de error que puede cometer un modelo, el equilibrio entre sesgo y varianza, y diferentes metodologías para evaluar el rendimiento de los modelos de forma robusta. Además, aplicaremos estas técnicas utilizando Python y la librería Scikit-Learn, brindando un enfoque práctico y aplicable al desarrollo profesional.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender los conceptos clave relacionados con el ajuste de un modelo de aprendizaje de máquina.
- Distinguir entre los diferentes tipos de error de un modelo y sus implicancias.
- Identificar y explicar los conceptos de sobreajuste, subajuste y ajuste apropiado.
- Interpretar el trade-off entre sesgo (bias) y varianza en la construcción de modelos.
- Explicar la importancia de la validación cruzada en el proceso de evaluación de modelos.

- Conocer y aplicar las principales técnicas de validación cruzada util.izadas en la industria

Nivel de Ajuste de un Modelo y Validación Cruzada

¿Qué es el ajuste de un modelo?

El ajuste de un modelo es el proceso mediante el cual se entrena un algoritmo de aprendizaje de máquina para que pueda aprender patrones a partir de un conjunto de datos. Este ajuste implica encontrar un equilibrio entre la capacidad del modelo para explicar los datos de entrenamiento y su habilidad para generalizar a nuevos datos. Un modelo bien ajustado es aquel que aprende lo suficiente de los datos para hacer predicciones precisas, evitando tanto el sobreajuste como el subajuste.

En términos simples, si el modelo es demasiado complejo, corre el riesgo de aprender también el "ruido" o las particularidades del conjunto de entrenamiento, lo que lleva a un mal rendimiento con datos nuevos. Por otro lado, si el modelo es demasiado simple, es posible que no capture la estructura subyacente de los datos, lo que también afecta negativamente su desempeño.

Uno de los elementos esenciales para medir el ajuste es la diferencia entre el rendimiento en los datos de entrenamiento y en los datos de prueba. Cuando esta diferencia es pequeña y ambos resultados son buenos, se dice que el modelo tiene un ajuste apropiado. Sin embargo, si hay una gran diferencia, esto puede ser una señal de problemas.

A lo largo del desarrollo de un modelo, es común observar tres posibles situaciones: sobreajuste (overfitting), subajuste (underfitting) o ajuste adecuado. Reconocer cada uno de estos estados es clave para optimizar el desempeño del modelo y lograr predicciones confiables.

1. Tipos de error de un modelo

Uno de los aspectos clave en el proceso de evaluación de modelos en Machine Learning es la comprensión de los distintos tipos de errores que un modelo puede cometer. Estos errores no solo reflejan el desempeño del modelo, sino que

también permiten identificar posibles problemas relacionados con el ajuste y la capacidad de generalización del modelo.

El error de un modelo se puede clasificar principalmente en dos categorías: **Error de entrenamiento** y **Error de prueba (o generalización)**. El primero mide el desempeño del modelo sobre los datos con los que fue entrenado, mientras que el segundo evalúa su rendimiento sobre datos no vistos durante el entrenamiento, permitiendo estimar su capacidad predictiva.

En términos generales, los errores que un modelo puede cometer son los siguientes:

- **Error sistemático o sesgo (Bias):** Ocurre cuando el modelo tiene suposiciones simplificadas sobre la relación entre las variables, lo que lleva a un desempeño pobre incluso en los datos de entrenamiento.

- **Error por varianza:** Se genera cuando el modelo es muy sensible a pequeñas variaciones en los datos de entrenamiento, resultando en una gran fluctuación del rendimiento en diferentes conjuntos de datos.

- **Error irreducible:** Proviene del ruido inherente a los datos y no puede ser eliminado por ningún modelo.

A continuación, se presenta una tabla resumen con los tipos de error:

Tipo de Error	Descripción	Ejemplo Práctico
Error de entrenamiento	Diferencia entre las predicciones del modelo y los valores reales en los datos de entrenamiento.	Modelo que no logra aprender patrones básicos.
Error de prueba	Diferencia entre las predicciones del modelo y los valores reales en datos no vistos.	Modelo que funciona bien en entrenamiento, pero falla en producción.
Error de sesgo	Error debido a supuestos erróneos o simplificados.	Un modelo lineal intentando predecir datos no lineales.
Error de varianza	Error debido a la sensibilidad extrema a los datos de entrenamiento.	Árbol de decisión con demasiada profundidad.

Error irreducible	Error asociado al ruido en los datos.	Datos con mediciones erróneas o inconsistencias.

Un aspecto relevante al analizar los errores es comprender que siempre existirán, pero el objetivo es reducirlos al máximo sin comprometer la capacidad de generalización del modelo.

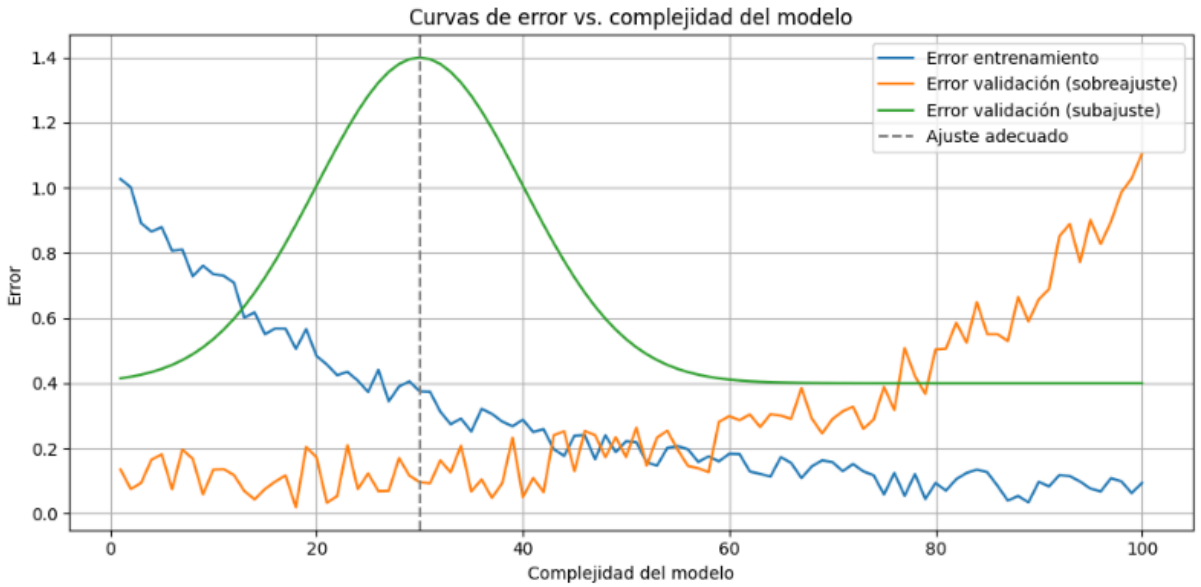
2. Sobreajuste, subajuste y ajuste adecuado

Cuando un modelo presenta **sobreajuste** significa que ha aprendido demasiado bien los datos de entrenamiento, incluyendo sus excepciones y ruidos. Esto se traduce en un bajo error de entrenamiento, pero un alto error en los datos de prueba. Los modelos muy complejos, como redes neuronales profundas o árboles de decisión sin poda, son susceptibles a este problema.

Por otro lado, el **subajuste** ocurre cuando el modelo es demasiado simple para capturar los patrones subyacentes de los datos. En este caso, tanto el error de entrenamiento como el error de prueba son altos. Un ejemplo clásico sería aplicar una regresión lineal para un problema claramente no lineal.

El objetivo siempre será lograr un **ajuste adecuado**, es decir, un modelo que no sea ni demasiado complejo ni demasiado simple. Esto permitirá un buen desempeño tanto en los datos de entrenamiento como en los datos de prueba, manteniendo un bajo error de generalización.

Detectar estos estados es posible mediante la observación de las curvas de error durante el entrenamiento y utilizando técnicas de validación adecuadas. El ajuste correcto se logra iterando sobre la complejidad del modelo y aplicando métodos de validación cruzada.



🧪 Ejemplo de código con validación cruzada

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression

# Generar datos simulados
X, y = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)

# Modelo simple
modelo = LinearRegression()

# Validación cruzada 5-fold
scores = cross_val_score(modelo, X, y, cv=5, scoring='neg_mean_squared_error')

# Mostrar resultados
print("Errores de validación cruzada (MSE):", -scores)
print("Error promedio:", -scores.mean())
```

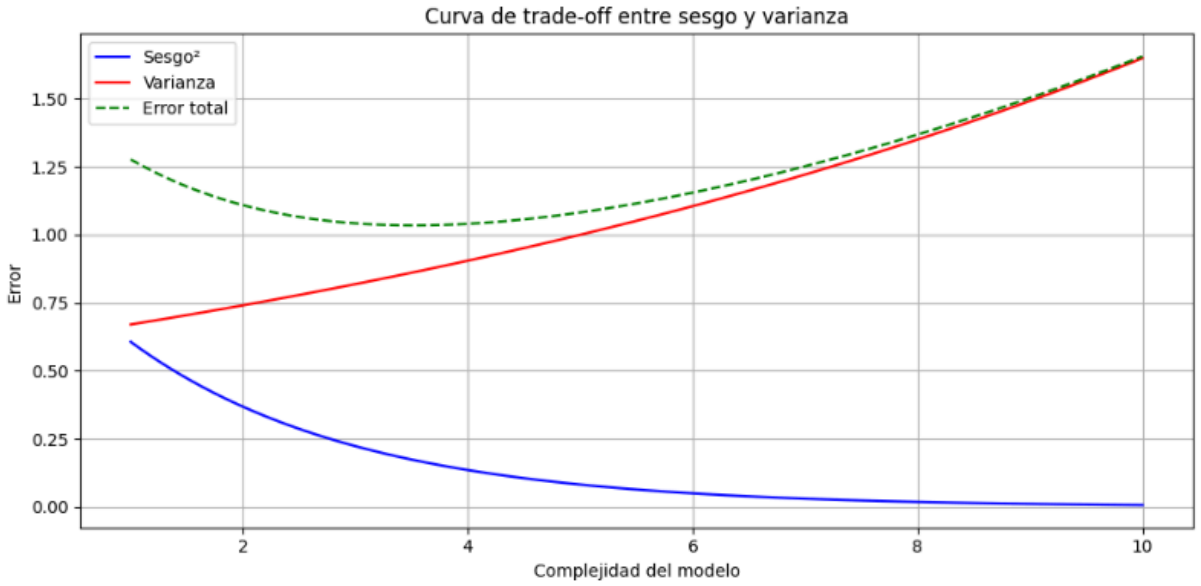
3. Trade-off entre sesgo y varianza

Uno de los conceptos clave en el ajuste de modelos es el **trade-off entre sesgo y varianza**. El **sesgo** se refiere a los errores introducidos por un modelo demasiado simple, incapaz de capturar las relaciones complejas en los datos. La **varianza**, en cambio, está asociada a un modelo muy complejo que se ajusta excesivamente a los datos de entrenamiento, siendo sensible a pequeñas variaciones.

Este equilibrio entre sesgo y varianza es representado gráficamente por la conocida **curva de sesgo-varianza**. Cuando el sesgo es alto, el modelo subestima la relación entre las variables; cuando la varianza es alta, el modelo sobreestima estas relaciones, afectando su capacidad de generalización.

La clave está en encontrar un punto intermedio donde ambos factores estén equilibrados, minimizando el error total. Esto se logra mediante la correcta selección del modelo, la complejidad adecuada y la implementación de técnicas de regularización o validación.

Comprender este trade-off es fundamental para los científicos de datos y analistas, ya que permite tomar decisiones informadas en el proceso de ajuste y evitar problemas que podrían comprometer la utilidad del modelo.



🧪 Ejemplo de código práctico con validación cruzada

```python
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import cross_val_score
import numpy as np

# Generar datos
X, y = make_regression(n_samples=100, n_features=20, noise=10, random_state=0)

# Modelo sin regularización (alta varianza posible)
modelo_simple = LinearRegression()
scores_simple = cross_val_score(modelo_simple, X, y, cv=5, scoring='neg_mean_squared_error')

# Modelo regularizado (menor varianza)
modelo_ridge = Ridge(alpha=10)
scores_ridge = cross_val_score(modelo_ridge, X, y, cv=5, scoring='neg_mean_squared_error')

print("Error promedio (LinearRegression):", -np.mean(scores_simple))
print("Error promedio (Ridge α=10):", -np.mean(scores_ridge))
```

4. ¿Qué es la validación cruzada?

La **validación cruzada** es una técnica estadística utilizada para evaluar el desempeño de un modelo y garantizar que los resultados obtenidos no sean producto de una partición específica de los datos. Consiste en dividir el conjunto de datos en múltiples subconjuntos, entrenando y evaluando el modelo varias veces con diferentes particiones.

El objetivo de la validación cruzada es obtener una estimación más robusta del error de generalización del modelo. Además, permite detectar posibles problemas de sobreajuste o subajuste, proporcionando una visión más realista del desempeño esperado.

Existen diversos métodos de validación cruzada, cada uno con sus ventajas y desventajas. La elección de un método específico depende del tamaño del conjunto de datos, la complejidad del modelo y los recursos computacionales disponibles.

La implementación adecuada de la validación cruzada es esencial en cualquier proyecto de aprendizaje de máquina, ya que mejora la confiabilidad de las conclusiones y evita tomar decisiones basadas en resultados engañosos.

5. Técnicas de validación cruzada

Introducción a la validación cruzada

La validación cruzada es una de las técnicas más utilizadas en el aprendizaje de máquina para evaluar la capacidad de generalización de un modelo. Su objetivo principal es asegurar que el modelo no se ajuste únicamente a los datos de entrenamiento, sino que también tenga un buen rendimiento con datos nuevos. A diferencia de una simple separación entrenamiento/prueba, la validación cruzada permite utilizar la totalidad de los datos para entrenar y probar el modelo de forma sistemática.

Esta técnica consiste en dividir el conjunto de datos en múltiples subconjuntos o "folds". Durante cada iteración, un fold es utilizado como conjunto de prueba y los demás como conjunto de entrenamiento. El proceso se repite tantas veces como folds existan y los resultados obtenidos se promedian para obtener una métrica más estable.

Una de las ventajas clave de la validación cruzada es que reduce la varianza de la estimación del rendimiento del modelo, ya que permite evaluar el modelo en diferentes subconjuntos de datos. Esto es especialmente útil cuando se dispone de un conjunto de datos limitado, ya que maximiza el uso de la información disponible.

Además, la validación cruzada ayuda a detectar problemas de sobreajuste o subajuste, brindando una perspectiva más precisa sobre la capacidad de generalización del modelo y facilitando la comparación entre diferentes algoritmos o configuraciones.

Método de Retención

El método de retención, también conocido como Hold-Out, es una de las técnicas más simples y utilizadas para validar un modelo de aprendizaje automático. Consiste en dividir el conjunto de datos en dos subconjuntos: uno para el entrenamiento del modelo y otro para la evaluación de su desempeño. Comúnmente, se utiliza un 70% u 80% de los datos para el entrenamiento y el porcentaje restante para la prueba.

Este método es sencillo de implementar y rápido de ejecutar, especialmente útil cuando se cuenta con un volumen grande de datos. La idea es que el conjunto de prueba simule datos nuevos y no vistos, permitiendo así obtener una estimación del rendimiento del modelo.

Sin embargo, el método de retención presenta limitaciones importantes. Una de las principales es que los resultados pueden depender excesivamente de cómo se haya realizado la partición de los datos. Si los datos no son representativos o contienen un sesgo, la evaluación será poco confiable.

Además, la partición simple no permite aprovechar al máximo los datos disponibles. Parte de la información útil para el entrenamiento se pierde al reservar un subconjunto exclusivamente para la evaluación. Esto es especialmente problemático cuando se trabaja con conjuntos de datos pequeños.

Por estas razones, el método de retención se recomienda principalmente para casos exploratorios, pruebas rápidas, o cuando se dispone de un volumen masivo de datos y no se requiere una evaluación refinada.

Finalmente, para reducir la variabilidad de los resultados, es posible repetir el método varias veces utilizando diferentes particiones y promediar los resultados, aunque esta práctica ya anticipa la lógica de técnicas más avanzadas como la validación cruzada.

6. Validación Cruzada de K-Iteraciones

La validación cruzada de k-iteraciones, también conocida como **K-Fold Cross Validation**, es una técnica sistemática que mejora la estimación de desempeño de un modelo y reduce el sesgo asociado a una única partición de los datos.

Esta técnica consiste en dividir el conjunto de datos en *k* subconjuntos o "folds" de tamaño similar. En cada iteración, uno de estos subconjuntos se utiliza como conjunto de prueba y los *k-1* restantes como conjunto de entrenamiento. Este proceso se repite *k* veces, de forma que cada subconjunto haya sido utilizado exactamente una vez como conjunto de prueba.

El valor de *k* es configurable, aunque los más comunes son 5 o 10. Un valor pequeño de *k* puede generar una alta varianza en la evaluación, mientras que un valor muy grande puede aumentar el coste computacional.

El principal beneficio de la validación cruzada de k-iteraciones es que reduce la dependencia de la evaluación respecto a un único conjunto de prueba. Además, permite aprovechar mejor el conjunto de datos, ya que cada observación es utilizada tanto para el entrenamiento como para la evaluación.

Entre sus desventajas, destaca el incremento en el tiempo de entrenamiento, ya que el modelo debe ser entrenado y evaluado *k* veces. Esto puede ser un factor limitante para modelos complejos o conjuntos de datos muy grandes.

A pesar de ello, es considerada una de las técnicas estándar y más fiables para evaluar modelos de Machine Learning, proporcionando un balance adecuado entre sesgo y varianza en la estimación de rendimiento.

7. Validación Cruzada Aleatoria

La validación cruzada aleatoria, también conocida como **Shuffle Split** o particionado aleatorio repetido, es una técnica que busca evaluar el desempeño del modelo mediante múltiples particiones aleatorias de los datos.

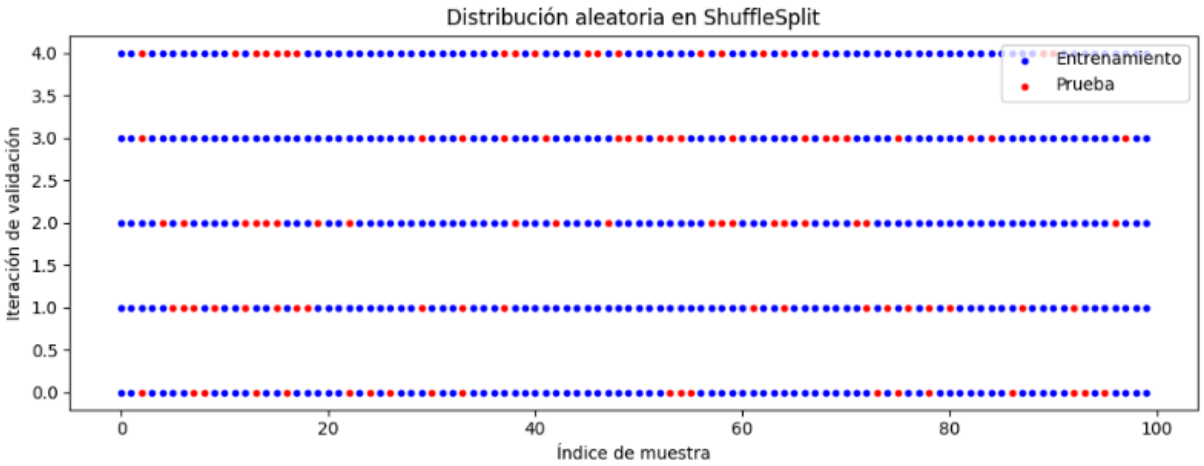
En este enfoque, se especifica un porcentaje para el conjunto de entrenamiento y otro para el conjunto de prueba. La división se realiza aleatoriamente varias veces, y el rendimiento final del modelo se obtiene promediando los resultados de todas las particiones.

Una de las ventajas clave de este método es la flexibilidad que ofrece al permitir controlar tanto el tamaño de los subconjuntos como la cantidad de particiones. Esto es útil para conjuntos de datos grandes, donde no es necesario utilizar todos los datos en cada iteración.

Sin embargo, la validación cruzada aleatoria también presenta desventajas. Puede darse el caso de que ciertos datos queden sin ser utilizados, o que algunos se repitan con frecuencia en diferentes particiones, lo que introduce cierto grado de dependencia entre las muestras.

Otra limitación es que no garantiza que todas las observaciones hayan sido utilizadas para la evaluación, a diferencia de la validación cruzada de k-iteraciones.

A pesar de estos aspectos, esta técnica resulta eficiente y recomendable cuando el volumen de datos es considerable y el costo computacional de una validación exhaustiva resulta prohibitivo.



🧪 Ejemplo de código con `ShuffleSplit` y `cross_val_score`

```
from sklearn.model_selection import ShuffleSplit, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
import numpy as np

# Dataset ficticio
X, y = make_classification(n_samples=500, n_features=10, random_state=42)

# Configurar validación cruzada aleatoria
shuffle = ShuffleSplit(n_splits=5, test_size=0.25, random_state=0)

# Modelo
modelo = LogisticRegression()

# Evaluar con cross_val_score
scores = cross_val_score(modelo, X, y, cv=shuffle, scoring='accuracy')

print("Scores por partición:", scores)
print("Precisión promedio:", np.mean(scores))
```

8. Validación Cruzada Dejando Uno Afuera

La validación cruzada dejando uno afuera, conocida como **Leave-One-Out Cross-Validation (LOOCV)**, es una técnica extrema de validación cruzada donde el número de iteraciones es igual al número de observaciones en el conjunto de datos.

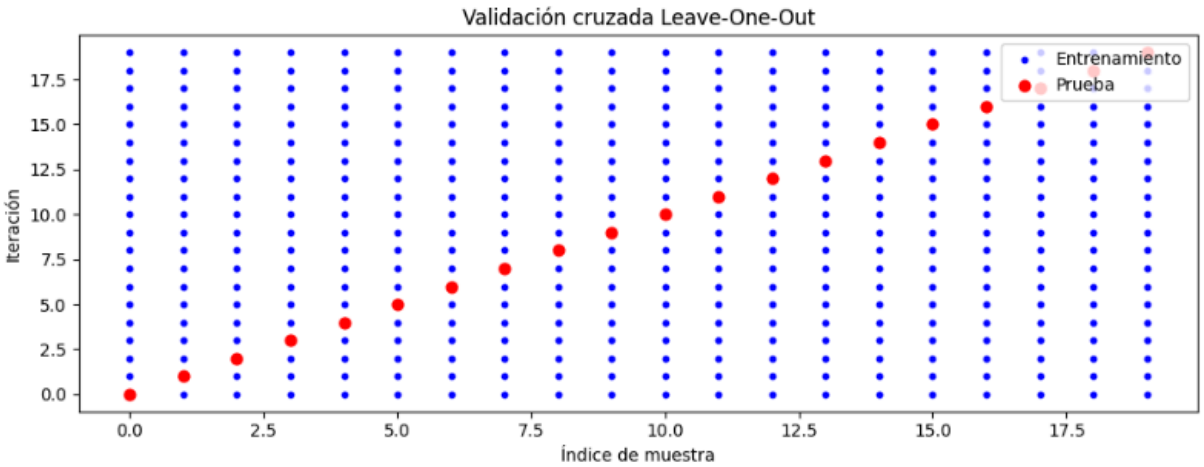
En cada iteración, se utiliza una sola observación como conjunto de prueba y el resto como conjunto de entrenamiento. Este proceso se repite para cada observación, asegurando que todas sean utilizadas como prueba una vez.

El principal beneficio de LOOCV es que maximiza el uso de los datos para el entrenamiento, minimizando el sesgo en la evaluación. Además, no requiere dividir arbitrariamente el conjunto de datos, evitando así problemas de particionado.

Sin embargo, su desventaja principal es el coste computacional, ya que requiere entrenar el modelo tantas veces como observaciones haya. Para conjuntos de datos grandes o modelos complejos, esto puede ser inviable.

Además, la varianza de la estimación de error puede ser alta, ya que cada modelo es validado sobre un solo dato, que puede ser atípico o poco representativo.

A pesar de ello, LOOCV es útil en contextos académicos o cuando se trabaja con conjuntos de datos pequeños, donde maximizar el uso de los datos es crucial.



🧪 Ejemplo práctico con `LeaveOneOut`

```python
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
import numpy as np

# Dataset pequeño
iris = load_iris()
X, y = iris.data, iris.target

# Seleccionamos solo dos clases para binarizar
X, y = X[y != 2], y[y != 2]

# Configuración LOOCV
loo = LeaveOneOut()

# Modelo
modelo = LogisticRegression(max_iter=1000)

# Evaluación
scores = cross_val_score(modelo, X, y, cv=loo, scoring='accuracy')
print("Precisión promedio con LOOCV:", np.mean(scores))
```

9. Implementación de Validación Cruzada con la Librería Scikit-Learn

Scikit-Learn, una de las bibliotecas más utilizadas en Machine Learning con Python, ofrece herramientas prácticas y eficientes para implementar las distintas técnicas de validación cruzada.

Para el método de retención, Scikit-Learn dispone de la función train_test_split, que permite dividir el conjunto de datos de forma sencilla, especificando el tamaño del conjunto de prueba y un estado aleatorio para reproducibilidad.

La validación cruzada de k-iteraciones puede implementarse fácilmente mediante la clase KFold o la función cross_val_score. Estas herramientas permiten definir el número de particiones y realizar la evaluación automática del modelo, devolviendo las métricas correspondientes.

La validación cruzada aleatoria está disponible mediante la clase ShuffleSplit, que permite definir tanto el número de divisiones como el tamaño relativo de los subconjuntos de entrenamiento y prueba, proporcionando gran flexibilidad.

Para la validación dejando uno afuera, Scikit-Learn ofrece la clase LeaveOneOut, que genera los índices necesarios para dividir los datos siguiendo esta técnica. También puede integrarse con cross_val_score para automatizar la evaluación.

Un ejemplo práctico básico de K-Fold Cross Validation sería:

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)
model = DecisionTreeClassifier()
kfold = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kfold)

print(f"Scores: {scores}")
print(f"Mean Accuracy: {scores.mean():.2f}")
```

Esta integración práctica permite evaluar de manera sistemática el rendimiento de los modelos, facilitando la comparación entre ellos y mejorando la toma de decisiones durante el proceso de modelado.

Cierre ✍️

En el desarrollo de un modelo de aprendizaje automático, comprender y gestionar el nivel de ajuste es fundamental para garantizar resultados precisos y generalizables. La correcta evaluación de los errores y el análisis del comportamiento del modelo frente a los datos de entrenamiento y prueba permiten identificar situaciones de sobreajuste, subajuste o ajuste adecuado, aspectos clave para la toma de decisiones informadas durante el proceso de modelado.

La validación cruzada emerge como una herramienta indispensable para medir la capacidad de generalización de los modelos. A través de diferentes técnicas, como el método de retención, la validación cruzada de k-iteraciones, la validación cruzada aleatoria y la validación dejando uno afuera, es posible obtener estimaciones más robustas y confiables del desempeño de los modelos, evitando que las evaluaciones dependan exclusivamente de una única partición de los datos.

Finalmente, la integración de estas técnicas en flujos de trabajo prácticos mediante librerías como Scikit-Learn facilita la implementación de procesos rigurosos de validación y comparación de modelos. La correcta aplicación de estas técnicas permite a los profesionales de datos construir modelos eficientes, capaces de resolver problemas reales y adaptarse adecuadamente a los desafíos del entorno de producción.

Referencias📚

- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., ... & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. Recuperado de https://scikit-learn.org/stable/user_guide.html

- Brownlee, J. (2020). *A gentle introduction to k-fold cross-validation*. Machine Learning Mastery. Recuperado de https://machinelearningmastery.com/k-fold-cross-validation/

- Wikipedia contributors. (2024, abril 24). *Cross-validation (statistics)*. Wikipedia. https://en.wikipedia.org/wiki/Cross-validation_(statistics)

- Scikit-learn developers. (2024). *sklearn.model_selection.LeaveOneOut*. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.LeaveOneOut.html

- Scikit-learn developers. (2024). *sklearn.model_selection.ShuffleSplit*. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.ShuffleSplit.html

- Scikit-learn developers. (2024). *Cross-validation: evaluating estimator performance*. scikit-learn. https://scikit-learn.org/stable/modules/cross_validation.html

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy